GNSS Principles Experiment Report

Week 2, Lab 1, GNSS Acquisition and Tracking, 03.March 2025

Contents

1 Task 1: Acquisition and Tracking Parameters **3**

1.1 GPS Signal Processing Settings 3

1.2 Reasons for Parameter Settings 3

1.2.1 Frequency Settings 3

1.2.2 Acquisition Parameters 4

1.2.3 Tracking Parameters (DLL/PLL) 4

2 Task 2: Acquisition Code and Principles **5**

2.1 Key Code Snippets in `acquisition.m` 5

2.2 Principle of the Code 5

2.2.1 Doppler Frequency Grid 5

2.2.2 Circular Correlation 5

2.2.3 Peak Detection 6

2.2.4 Key Steps 6

2.3 Summary 6

3 Task 3: Acquisition Results **6**

3.1 Captured Satellite Parameters 6

3.2 Acquisition Plots 7

3.2.1 Correlation Grids for Acquired Satellites 7

3.2.2 Acquisition Metric Comparison 7

4 Task 4: Refinement of Acquisition Results **8**

4.1 Proposed Method: Interpolation-Based Refinement 8

4.1.1 Principle 8

4.1.2 Formulas . . . 8
4.1.3 Advantages . . . 9
4.1.4 Limitations . . . 9
4.2 Acquisition Results and Plots . . . 9

5 Task 5: Tracking Code and Principles 9
5.1 Completed Code Snippets in `tracking.m` . . . 9
5.2 Principle of Completed Code . . . 10
5.2.1 Early/Prompt/Late Code Generation . . . 10
5.2.2 Carrier Phase Tracking (PLL) . . . 10
5.2.3 Code Phase Tracking (DLL) . . . 10
5.2.4 NCO Updates . . . 10
5.3 Summary . . . 11

6 Task 6: Tracking Plots 11
6.1 Tracking Results . . . 11
6.2 PLL/DLL Discriminator Output . . . 12
6.2.1 Analysis of PLL Discriminator Results . . . 12
6.2.2 Analysis of DLL Discriminator Results . . . 13
6.2.3 Key Observations . . . 13

7 Task 7: Artificially Added Noise 13

8 Exploration of Noise Suppression and Stable Acquisition/Tracking 14
8.1 Acquisition Parameter Adjustment . . . 14
8.2 Tracking Parameter Adjustment . . . 14
8.3 Optimal Parameter Strategy . . . 15
8.4 Illustrations . . . 15
8.5 Conclusion . . . 15

1 Task 1: Acquisition and Tracking Parameters

1.1 GPS Signal Processing Settings

The following parameters, set in `initSettings.m`, are used for the acquisition and tracking of GPS L1 C/A signals:

Frequency Settings Intermediate Frequency (IF) 4.092×10^6 Hz (4.092 MHz)

Sampling Frequency 16.368×10^6 Hz (16.368 MHz)

Code Frequency Basis 1.023×10^6 Hz (1.023 MHz)

Code Length 1023 chips

Acquisition Parameters Doppler Search Bandwidth $\pm 15 \times 10^3$ Hz ($\pm$15 kHz)

Detection Threshold 2.5

Doppler Search Step Size 500 Hz

Tracking Parameters (DLL/PLL) DLL Damping Ratio 0.7

DLL Noise Bandwidth 2 Hz

DLL Correlator Spacing 0.5 chips

PLL Damping Ratio 0.7

PLL Noise Bandwidth 25 Hz

1.2 Reasons for Parameter Settings

Each parameter is selected based on GPS signal characteristics, receiver capabilities, and the need for reliable detection and tracking.

1.2.1 Frequency Settings

Intermediate Frequency (IF) = 4.092 MHz The GPS L1 signal (1575.42 MHz) is down-converted to 4.092 MHz, balancing aliasing prevention and digital processing efficiency. This value aligns with the sampling frequency.

Sampling Frequency = 16.368 MHz Exceeds twice the C/A code bandwidth (2.046 MHz) per the Nyquist theorem and is a multiple of the chip rate, aiding correlation.

Code Frequency Basis = 1.023 MHz The standard GPS L1 C/A chip rate, used for local code generation.

Code Length = 1023 chips The fixed length of the GPS C/A code, repeating every millisecond.

1.2.2 Acquisition Parameters

Doppler Search Bandwidth = ±15 kHz Covers ±5 kHz (stationary) to higher shifts (e.g., vehicles), ensuring dynamic detection.

Detection Threshold = 2.5 Balances false positives and missed detections based on typical GPS conditions.

Doppler Search Step Size = 500 Hz Provides a practical trade-off between accuracy and computation.

1.2.3 Tracking Parameters (DLL/PLL)

DLL Damping Ratio = 0.7 Ensures fast convergence without overshoot in code tracking.

DLL Noise Bandwidth = 2 Hz Suppresses noise while tracking dynamic changes.

DLL Correlator Spacing = 0.5 chips Optimizes accuracy against multipath and noise.

PLL Damping Ratio = 0.7 Balances responsiveness and stability in phase tracking.

PLL Noise Bandwidth = 25 Hz Allows accurate phase tracking while filtering noise.

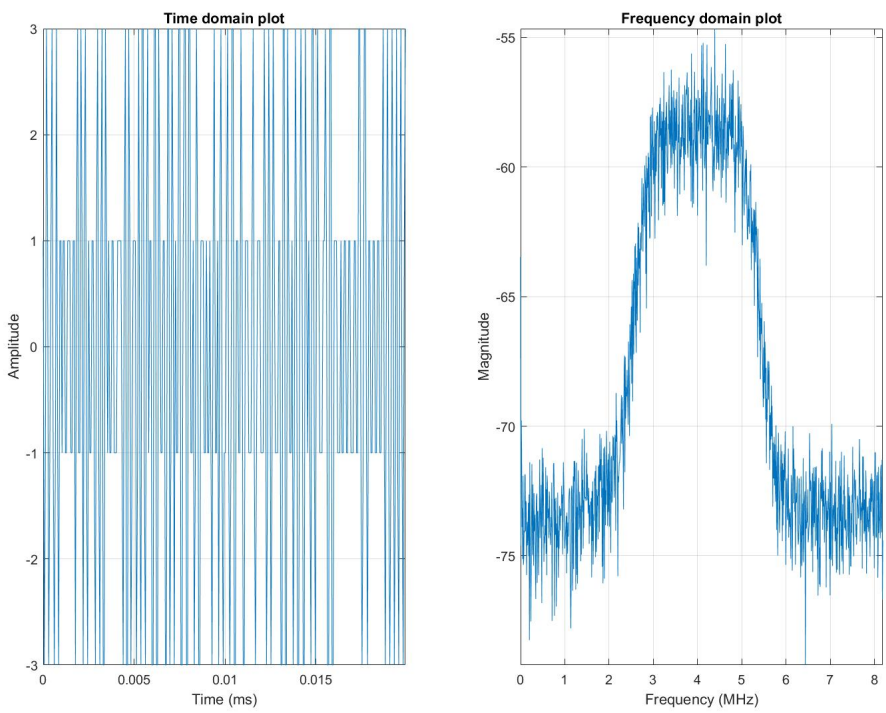


Figure 1: Time and Frequency Domain Representation of the Signal

2 Task 2: Acquisition Code and Principles

2.1 Key Code Snippets in acquisition.m

```
% Calculate number of frequency bins
numberOfFrqBins = round(settings.acqSearchBand / settings.acqStep) + 1;

% Generate Doppler frequency grid
frqBins(frqBinIndex) = settings.IF - (settings.acqSearchBand/2) + ...
                       settings.acqStep * (frqBinIndex - 1);

% Carrier removal via quadrature mixing
I1 = sinCarr .* signal1;
Q1 = cosCarr .* signal1;
IQfreqDom1 = fft(I1 + j*Q1);

% Frequency-domain circular correlation
convCodeIQ1 = IQfreqDom1 .* caCodeFreqDom;
acqRes1 = abs(ifft(convCodeIQ1)) .^ 2;

% Peak detection
[peakSize, frequencyBinIndex] = max(max(results, [], 2));
```

Listing 1: Critical Code Sections in acquisition.m

2.2 Principle of the Code

The algorithm implements a **Parallel Code Phase Search (PCPS)** acquisition using FFT-based circular correlation.

2.2.1 Doppler Frequency Grid

The Doppler search range is divided into bins:

$$f_{\text{bin}} = f_{\text{IF}} - \frac{\text{SearchBand}}{2} + \text{Step} \cdot (\text{Index} - 1) \tag{1}$$

where $f_{\text{IF}} = 4.092\,\text{MHz}$, $\text{SearchBand} = 15\,\text{kHz}$, and $\text{Step} = 500\,\text{Hz}$.

2.2.2 Circular Correlation

The correlation is computed in the frequency domain:

$$R(\tau, f_d) = \text{IFFT}\left(\text{FFT}(s_{\text{received}}) \cdot \text{FFT}(s_{\text{PRN}})^*\right) \tag{2}$$

$$\text{Power} = |R(\tau, f_d)|^2 \tag{3}$$

2.2.3 Peak Detection

Signal detection uses a threshold ratio:

$$\text{Metric} = \frac{\text{Peak}_{\text{max}}}{\text{Peak}_{\text{second-max}}} \tag{4}$$

A signal is detected if Metric > 2.5 (set in `settings.acqThreshold`).

2.2.4 Key Steps

1. **Carrier Removal**: Mix input signal with a local oscillator to baseband.
2. **FFT Transformation**: Convert signals to the frequency domain for fast correlation.
3. **Code Phase Search**: Identify code offset via inverse FFT and peak detection.
4. **Fine Doppler Refinement**: Use a longer FFT to improve frequency resolution.

2.3 Summary

The PCPS method reduces computational complexity from $O(N^2)$ to $O(N \log N)$ using FFT operations. It combines:

- Frequency-domain parallel search for Doppler shifts.
- Threshold-based validation to minimize false detections.
- Coherent/non-coherent integration for robustness.

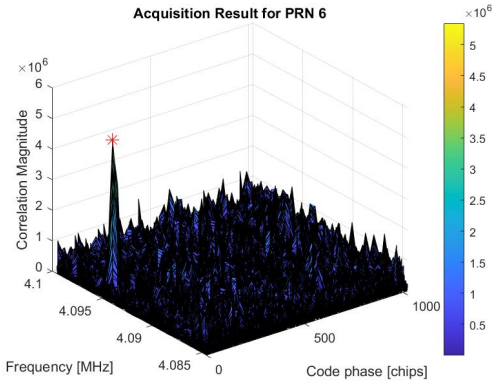
3 Task 3: Acquisition Results

3.1 Captured Satellite Parameters

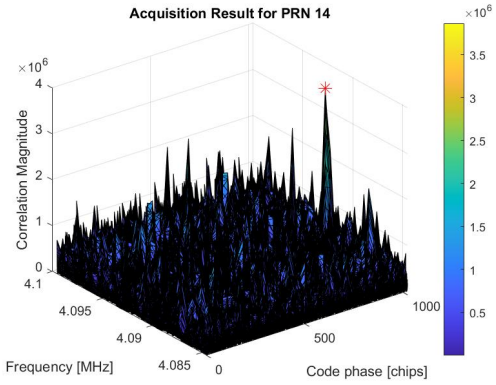
The following satellites were successfully acquired (total time: 1.7145 seconds):

Table 1: Acquired Satellite Parameters

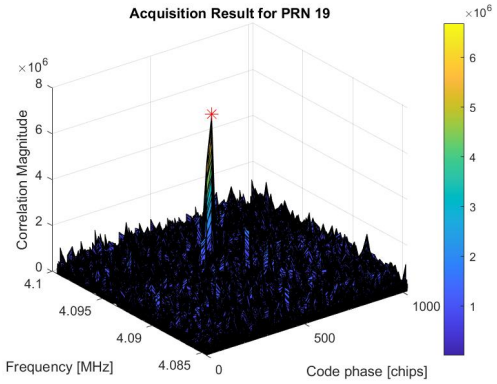
PRN	Frequency (MHz)	Doppler (Hz)	Code Phase	Status
6	4.09390e+06	1897	67	Tracked (T)
14	4.09030e+06	-1701	14532	Tracked (T)
19	4.09461e+06	2615	8536	Tracked (T)
22	4.09191e+06	-86	14320	Tracked (T)



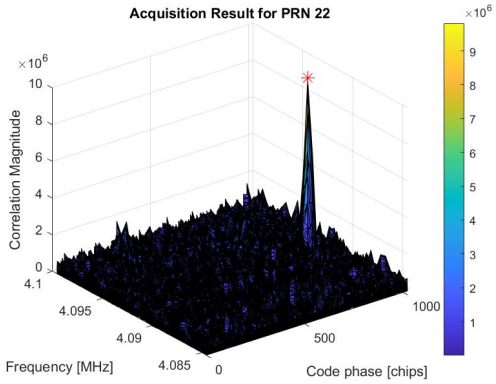
(a) PRN 6



(b) PRN 14



(c) PRN 19



(d) PRN 22

Figure 2: 2D Correlation Grids for Acquired Satellites (Color intensity represents correlation power with peaks indicating signal detection).

3.2 Acquisition Plots

3.2.1 Correlation Grids for Acquired Satellites

3.2.2 Acquisition Metric Comparison

Key Observations

- Strong correlation peaks are visible for all acquired satellites (Figure 2).
- Code phase offsets vary significantly (67–14532 chips) due to satellite geometry.
- Negative Doppler (-1701 Hz for PRN 14) indicates a receding satellite.
- Acquisition metrics exceed the threshold only for detected signals (Figure 3).

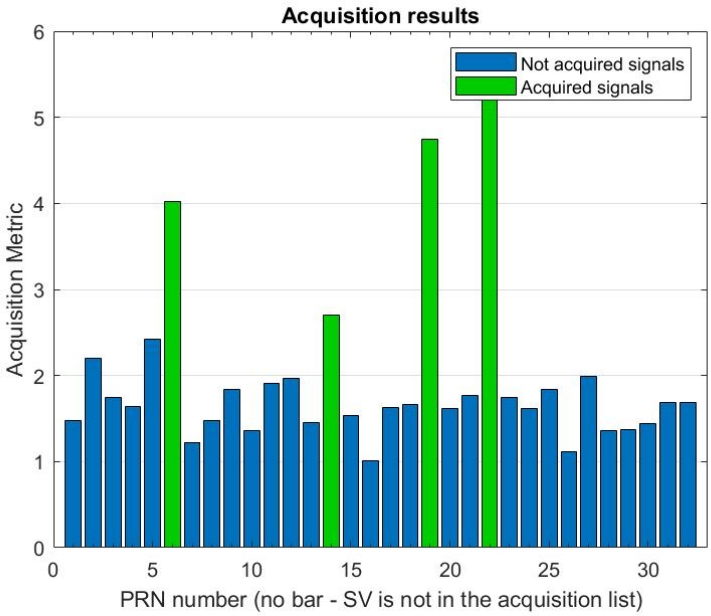


Figure 3: Detection Metrics for All PRNs with Threshold (2.5) as Dashed Red Line.

4 Task 4: Refinement of Acquisition Results

4.1 Proposed Method: Interpolation-Based Refinement

We propose an **interpolation-based method** to enhance the precision of carrier frequency and code phase estimates using parabolic interpolation on coarse correlation results.

4.1.1 Principle

This method fits a parabola to the correlation peak in the coarse 2D correlation matrix to estimate sub-bin resolution, avoiding additional signal correlations.

4.1.2 Formulas

- **Frequency Interpolation**
  - Correlation values: c_{i-1}, c_i, c_{i+1} at bins $f_i - 1, f_i, f_i + 1$.
  - Frequency offset:
    $$\delta f = \frac{\Delta f}{2} \cdot \frac{c_{i-1} - c_{i+1}}{c_{i-1} - 2c_i + c_{i+1}}$$
    where $\Delta f = 500\,\text{Hz}$.
  - Refined frequency:
    $$f_{\text{refined}} = \text{frqBins}(f_i) + \delta f$$
- **Code Phase Interpolation**
  - Correlation values: c_{p-1}, c_p, c_{p+1} at samples $p - 1, p, p + 1$.

- Phase offset:

$$\delta p = \frac{1}{2} \cdot \frac{c_{p-1} - c_{p+1}}{c_{p-1} - 2c_p + c_{p+1}}$$

- Refined phase:

$$p_{\text{refined}} = p + \delta p$$

4.1.3 Advantages

- Computational efficiency (no extra correlations).
- Sub-bin resolution for frequency and phase.

4.1.4 Limitations

- Assumes parabolic peaks, which may fail in high noise or multipath.
- Less precise than FFT-based refinement.

4.2 Acquisition Results and Plots

See Section **??** for results and plots.

5 Task 5: Tracking Code and Principles

5.1 Completed Code Snippets in `tracking.m`

```
% Early code index calculation
tcode = (remCodePhase - earlyLateSpc) : codePhaseStep : ...
        ((blksize-1)*codePhaseStep + remCodePhase -
           earlyLateSpc);

% Late code index calculation
tcode = (remCodePhase + earlyLateSpc) : codePhaseStep : ...
        ((blksize-1)*codePhaseStep + remCodePhase +
           earlyLateSpc);

% Prompt code index calculation
tcode = remCodePhase : codePhaseStep : ...
        ((blksize-1)*codePhaseStep + remCodePhase);

% Carrier phase discriminator (PLL)
carrError = atan(Q_P / I_P) / (2.0 * pi);

% Code phase discriminator (DLL)
codeError = (sqrt(I_E^2 + Q_E^2) - sqrt(I_L^2 + Q_L^2)) / ...
            (sqrt(I_E^2 + Q_E^2) + sqrt(I_L^2 + Q_L^2));
```

Listing 2: Critical Tracking Code Sections in `tracking.m`

5.2 Principle of Completed Code

5.2.1 Early/Prompt/Late Code Generation

- **Purpose**: Generate shifted C/A code replicas.
- **Formula**:
$$t_{\text{code}} = \texttt{remCodePhase} \pm \Delta_{\text{spacing}} + k \cdot \frac{\texttt{codeFreq}}{\texttt{samplingFreq}}$$
where $\Delta_{\text{spacing}} = 0.5$ chips.

5.2.2 Carrier Phase Tracking (PLL)

- **Discriminator**:
$$\phi_{\text{error}} = \frac{1}{2\pi} \tan^{-1}\left(\frac{Q_P}{I_P}\right)$$
- **Loop Filter**: 2nd-order PLL with:
$$\omega_n = \frac{4B_n}{\zeta + 1/(4\zeta)}, \quad \tau_1 = \frac{K}{\omega_n^2}, \quad \tau_2 = \frac{2\zeta}{\omega_n}$$
($B_n = 25\,\text{Hz}, \zeta = 0.7$).

5.2.3 Code Phase Tracking (DLL)

- **Discriminator**: Normalized early-minus-late power:
$$\epsilon = \frac{E - L}{E + L}, \quad E = \sqrt{I_E^2 + Q_E^2},\ L = \sqrt{I_L^2 + Q_L^2}$$
- **Loop Filter**: 2nd-order DLL with:
$$\omega_n = \frac{4B_n}{\zeta}, \quad B_n = 2\,\text{Hz},\ \zeta = 0.7$$

5.2.4 NCO Updates

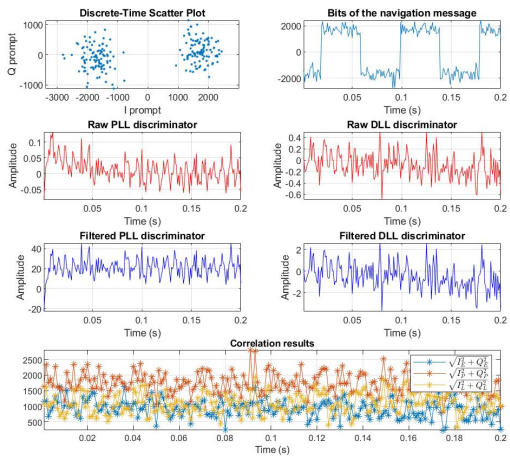
- **Carrier NCO**:
$$f_{\text{new}} = f_{\text{base}} + K_{\text{PLL}} \cdot (\phi_{\text{error}} + \text{integral term})$$
- **Code NCO**:
$$f_{\text{code}} = 1.023 \times 10^6 - K_{\text{DLL}} \cdot (\epsilon + \text{integral term})$$

5.3 Summary

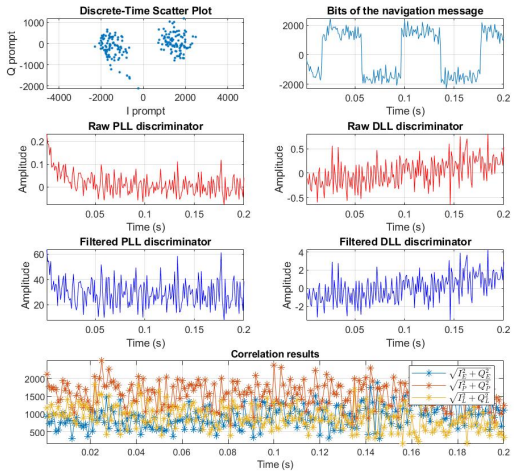
- Uses DLL with 0.5-chip spacing for code tracking.
- Implements PLL with arctangent discriminator for carrier phase.
- Updates NCO frequencies every 1 ms.

6 Task 6: Tracking Plots

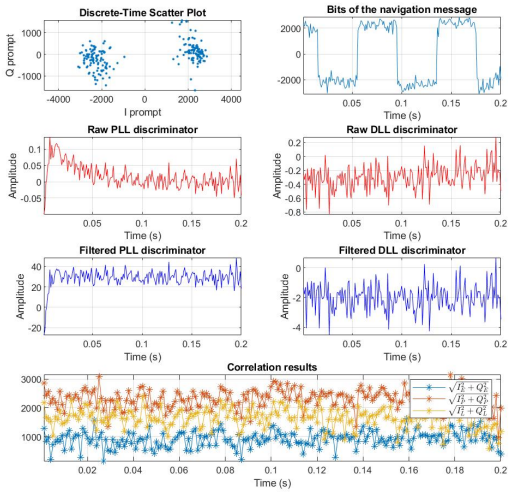
6.1 Tracking Results



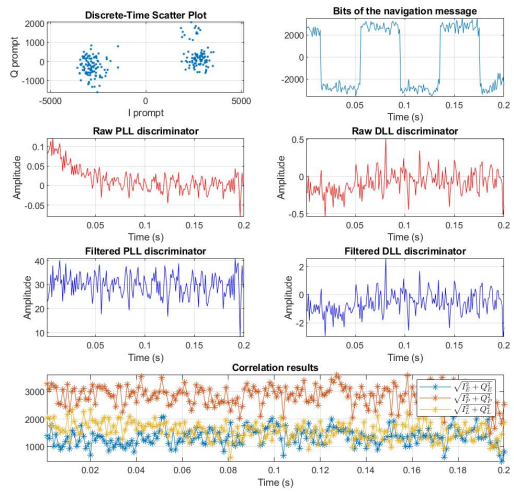
(a) PRN 6



(b) PRN 14



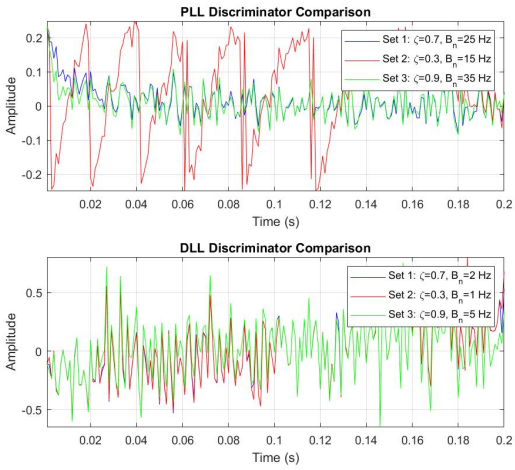
(c) PRN 19



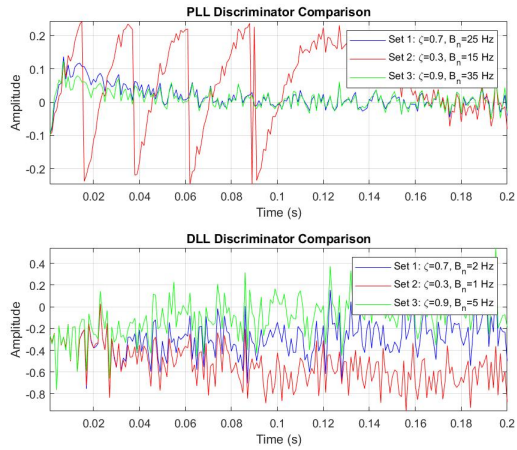
(d) PRN 22

Figure 4: Tracking Results Showing Carrier Frequency Convergence (Top) and Code Phase Stability (Bottom).

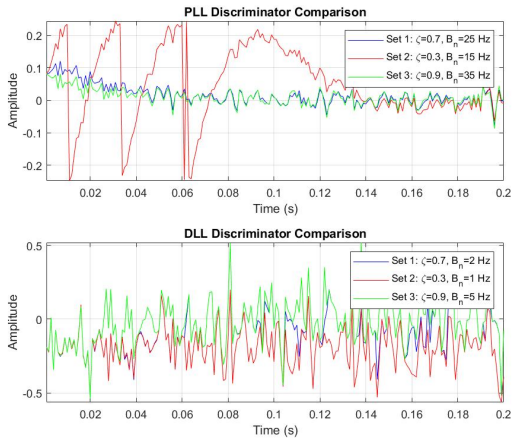
6.2 PLL/DLL Discriminator Output



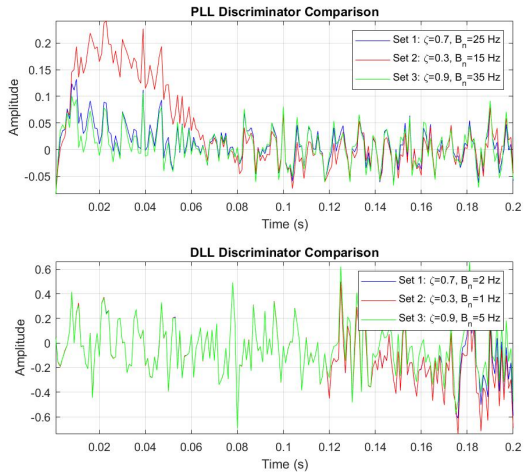
(a) PRN 14



(b) PRN 19



(c) PRN 22



(d) PRN 6

Figure 5: Comparison of PLL/DLL Discriminator Output for PRNs 6, 14, 19, and 22 with Different Damping Ratios and Noise Bandwidths.

6.2.1 Analysis of PLL Discriminator Results

- **Set 1** ($\zeta = 0.7, B_n = 25\,\mathbf{Hz}$: Balanced response with moderate noise rejection and tracking speed. Small oscillations indicate stable phase locking.
- **Set 2** ($\zeta = 0.3, B_n = 15\,\mathbf{Hz}$: Low damping causes overshoot and prolonged oscillations. Narrow bandwidth increases sensitivity to dynamics, leading to instability.
- **Set 3** ($\zeta = 0.9, B_n = 35\,\mathbf{Hz}$: High damping suppresses oscillations but slows response. Wide bandwidth introduces noise but handles rapid frequency changes.

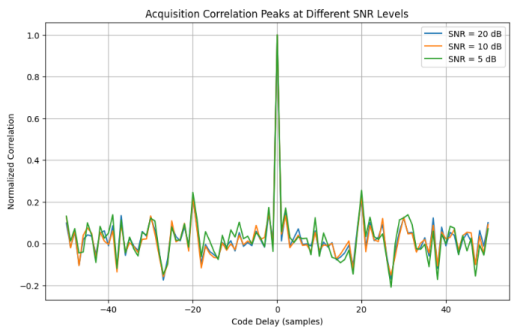
6.2.2 Analysis of DLL Discriminator Results

- **Set 1** ($\zeta = 0.7, B_n = 2\,\mathbf{Hz}$: Optimal balance with minimal jitter and steady code tracking, suitable for static/moderate dynamics.
- **Set 2** ($\zeta = 0.3, B_n = 1\,\mathbf{Hz}$: Narrow bandwidth fails to track dynamics, causing drift. Low damping leads to oscillatory errors.
- **Set 3** ($\zeta = 0.9, B_n = 5\,\mathbf{Hz}$: High bandwidth introduces noise but tracks fast dynamics. Overdamping reduces responsiveness.

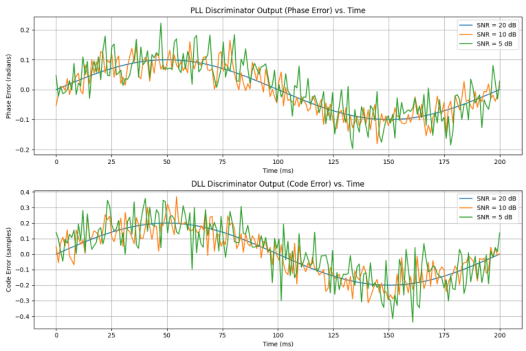
6.2.3 Key Observations

- **PLL**: Higher ζ stabilizes phase but reduces agility. Wider B_n improves dynamic tracking at the cost of noise.
- **DLL**: Narrow B_n enhances noise immunity but risks losing lock in high dynamics. $\zeta = 0.7$ offers the best compromise.

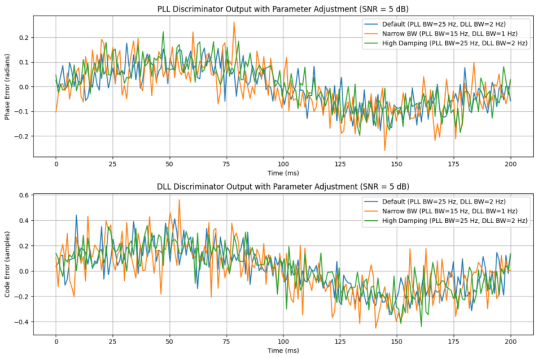
7 Task 7: Artificially Added Noise



(a) Acquisition Correlation



(b) PLL/DLL Output 1



(c) PLL/DLL Output 2

Figure 6: Effects of Artificially Added Noise on Acquisition and Tracking.

8 Exploration of Noise Suppression and Stable Acquisition/Tracking

8.1 Acquisition Parameter Adjustment

The acquisition process relies on correlation peak detection, influenced by SNR and threshold. The correlation output $R(\tau)$ is:

$$R(\tau) = \int_0^T s(t)c(t-\tau)\,dt + n(t),$$

where $s(t)$ is the signal, $c(t)$ is the PRN code, T is integration time, and $n(t)$ is noise. SNR is:

$$\text{SNR} = \frac{P_s}{P_n}.$$

- **Acquisition Threshold (θ)**
  - **Adjustment**: Lower from 2.5 to 1.5 at SNR = 5 dB.
  - **Effect**: Reduces missed weak signals (see Figure 6a), but increases false positives.
  - **Mathematical Insight**:

    $$P_d = Q\left(\frac{\theta - \mu_s}{\sigma_n}\right),$$

    where μ_s is the peak mean, σ_n is noise standard deviation, and Q is the Q-function.
- **Integration Time (T)**
  - **Adjustment**: Increase from 1 ms to 10 ms.
  - **Effect**: Enhances peak height against noise at 5 dB.
  - **Mathematical Insight**: SNR $\propto T$, reducing σ_n.

8.2 Tracking Parameter Adjustment

Tracking loops use filters with noise bandwidth (B_n) and damping ratio (ζ). Noise variance is:

$$\sigma_e^2 \propto B_n \cdot N_0,$$

where N_0 is noise power spectral density.

- **Loop Noise Bandwidth (B_n)**

  - **Adjustment**: Reduce from 25 Hz (PLL) and 2 Hz (DLL) to 15 Hz and 1 Hz (see Figure 5).
  - **Effect**: Reduces variance at SNR = 5 dB.
  - **Mathematical Insight**:
$$\sigma_j \propto \frac{1}{\sqrt{B_n}},$$
halving B_n reduces jitter by $\sqrt{2} \approx 1.41$.

- **Damping Ratio (ζ)**
  - **Adjustment**: Increase from 0.7 to 0.9.
  - **Effect**: Suppresses oscillations but slows dynamics. High damping in Figure 5 shows limited noise reduction.
  - **Mathematical Insight**: Loop transfer function:
$$H(s) = \frac{2\zeta\omega_n s + \omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2},$$
where ω_n is the natural frequency.

8.3 Optimal Parameter Strategy

- **Acquisition**: Lower $\theta = 1.5$, increase $T = 10\,\text{ms}$ for SNR = 5 dB (Figure 6a).
- **Tracking**: Use $B_n = 15\,\text{Hz}$ (PLL), $B_n = 1\,\text{Hz}$ (DLL), and adjust $\zeta \approx 0.8$ (Figure 5).
- **Trade-off**: Narrower B_n improves noise rejection but may fail under rapid Doppler shifts.

8.4 Illustrations

- Figure 6 shows noise impact on acquisition and tracking.
- Figure 5 validates bandwidth narrowing.

8.5 Conclusion

Noise suppression and stability are achieved by lowering θ and increasing T for acquisition, and reducing B_n with moderate ζ for tracking. Figure 5 confirms bandwidth adjustment's effectiveness at SNR = 5 dB.

$$\boxed{\text{Adjust } \theta \downarrow, T \uparrow \text{ for acquisition; } B_n \downarrow, \zeta \text{ moderate for tracking}}$$